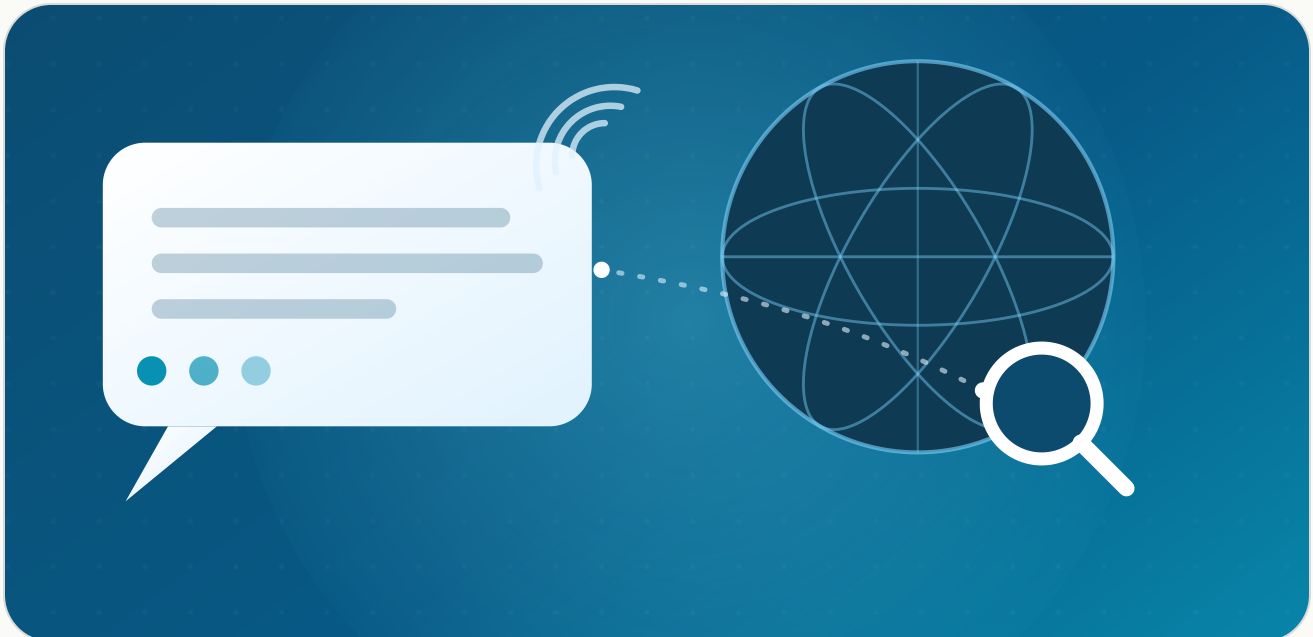


CASE STUDY

Conversational Agent with Web Search

A minimal LangGraph agent — two nodes, one conditional edge — that decides for itself when to search the live web.

- Functional end-to-end: streaming chat, tool-augmented search, and multi-turn memory all work. No automated tests, auth, or production hardening yet — a working prototype, not a hardened service.



01 Overview

Plexa is a from-scratch reimplementation of the "ask a question, get a web-grounded answer" pattern using LangGraph instead of a prebuilt agent framework. The entire decision loop — answer directly, or search first? — is two graph nodes and one conditional edge. There's no separate classification step: GPT-4o decides whether to call the Tavily search tool through native OpenAI tool-calling, and LangGraph's router just reads that decision and routes accordingly.

02 The Problem

A model's training cutoff means it can't answer questions about anything that happened after it was trained, and users notice immediately when a chat assistant confidently gets current events wrong. The fix doesn't need to be complicated — it needs the model to recognize when it's out of its depth and pull in a live source before answering, then stream the result back so the wait doesn't feel like a hang.

03 Design Principles

- Let the model decide, don't hardcode a router — GPT-4o is bound to a single tool (Tavily search); whether to call it is a model

decision made through native function-calling, not an if/else on keywords.

- State is just the message list — LangGraph's `add_messages` annotation accumulates conversation history automatically; there's no separate context-builder module to keep in sync with it.
- Stream the reasoning, not just the answer — the client receives a `search_start` event carrying the actual query the model chose to run, so a user can see when and why a search happened, not just wait for a final blob of text.

04 Architecture Deep-Dive

How the 8 pipeline stages (see the project overview) group into four defensive layers.

01 Decide

MODEL NODE

A single node wraps `ChatOpenAI(model="gpt-4o")` with `bind_tools([TavilySearchResults(max_results=4)])`. On every turn, the model either answers directly or emits a tool call — there's no separate classification prompt or keyword heuristic sitting in front of it; tool selection is native to GPT-4o's function-calling.

02 Search

TOOLS_ROUTER + TOOL_NODE

tools_router inspects the model's last message: if it contains tool calls, execution goes to tool_node; otherwise the graph ends. tool_node runs the Tavily call, wraps the result in a ToolMessage, and routes back to model — so the model sees the search results as part of the same message history on its next pass, with no manual context-stitching required.

03 **Stream & Remember**

SSE STREAMING + MEMORYSAVER CHECKPOINTING

Responses stream token-by-token over Server-Sent Events by consuming graph.astream_events and translating LangGraph's internal event types — on_chat_model_stream, on_tool_end, and friends — into a small SSE protocol the Next.js client understands: content chunks, a search_start signal, and a search_results event carrying just the source URLs. Multi-turn memory comes from LangGraph's MemorySaver, keyed by a checkpoint_id the client stores and replays on the next message — no database, no session table.

05 **Production & Compliance**

The backend is Dockerized and documented for deployment on Render; the Next.js frontend deploys separately. A few things are prototype-grade by choice, not accident: CORS currently allows all origins, there's no authentication or rate limiting, and conversation memory lives in LangGraph's in-memory checkpointer — it resets on

server restart rather than persisting anywhere. There's also a single LLM provider (GPT-4o); unlike the medication-reconciliation project, there's no fallback model if OpenAI is degraded.

Deployment

Dockerized backend, documented for Render; Next.js frontend deployed separately

Realtime

Server-Sent Events (astream_events)
— no WebSocket layer

Search

Tavily Search API, model-triggered

Llm

GPT-4o via native OpenAI tool-calling, single provider (no fallback)

Cache

None — LangGraph's in-memory MemorySaver only, resets on server restart

Monitoring

None yet — no test suite or usage analytics wired up

06 Results

There's no formal benchmark suite backing specific latency numbers here — unlike the medication-reconciliation project's 100-run baseline, this one hasn't been load-tested. What's concretely true from the code: streaming works over SSE with visible time-to-first-token, tool-calling correctly routes between direct answers and search-augmented ones, and multi-turn memory persists correctly within a server session via the checkpoint.

- Real-time SSE streaming, token by token
- Native GPT-4o tool-calling — no hardcoded query classifier
- Multi-turn memory via LangGraph MemorySaver checkpointing
- Live web search grounding via Tavily when the model decides it's needed
- Dockerized backend, deployment documented for Render
- Minimal 2-node graph — model + tool_node behind one conditional edge

07 What's Next

The gaps are the ones you'd expect from a prototype: no automated test suite yet (the medication-reconciliation project has 13; this one has none), no persistent conversation storage, no fallback LLM provider if OpenAI has an outage, and CORS/auth that need locking down before this could sit behind a real user base. On the product side, surfacing citations inline in the response text — not just as a source list — would close the gap with how Perplexity itself presents grounded answers.

08 Tech Stack

Python

FastAPI

LangGraph

LangChain

GPT-4o

Tavily Search API

TypeScript

Next.js

React

Server-Sent Events

Docker